

МИНОБРНАУКИ РОССИИ

Федеральное государственное бюджетное образовательное учреждение высшего
образования



НИЖЕГОРОДСКИЙ ГОСУДАРСТВЕННЫЙ ТЕХНИЧЕСКИЙ

УНИВЕРСИТЕТ им. Р.Е.АЛЕКСЕЕВА

Институт радиоэлектроники и информационных технологий

Кафедра «Вычислительные системы и технологии»

ОТЧЕТ

по лабораторной работе №3

по дисциплине

Языки интернет-программирования

РУКОВОДИТЕЛЬ:

(подпись)

Яшнов М.Д.
(фамилия, и.,о.)

СТУДЕНТ:

(подпись)

Рыжов А.С.
(фамилия, и.,о.)

23-ИИ
(шифр группы)

Работа защищена «__» _____

С оценкой _____

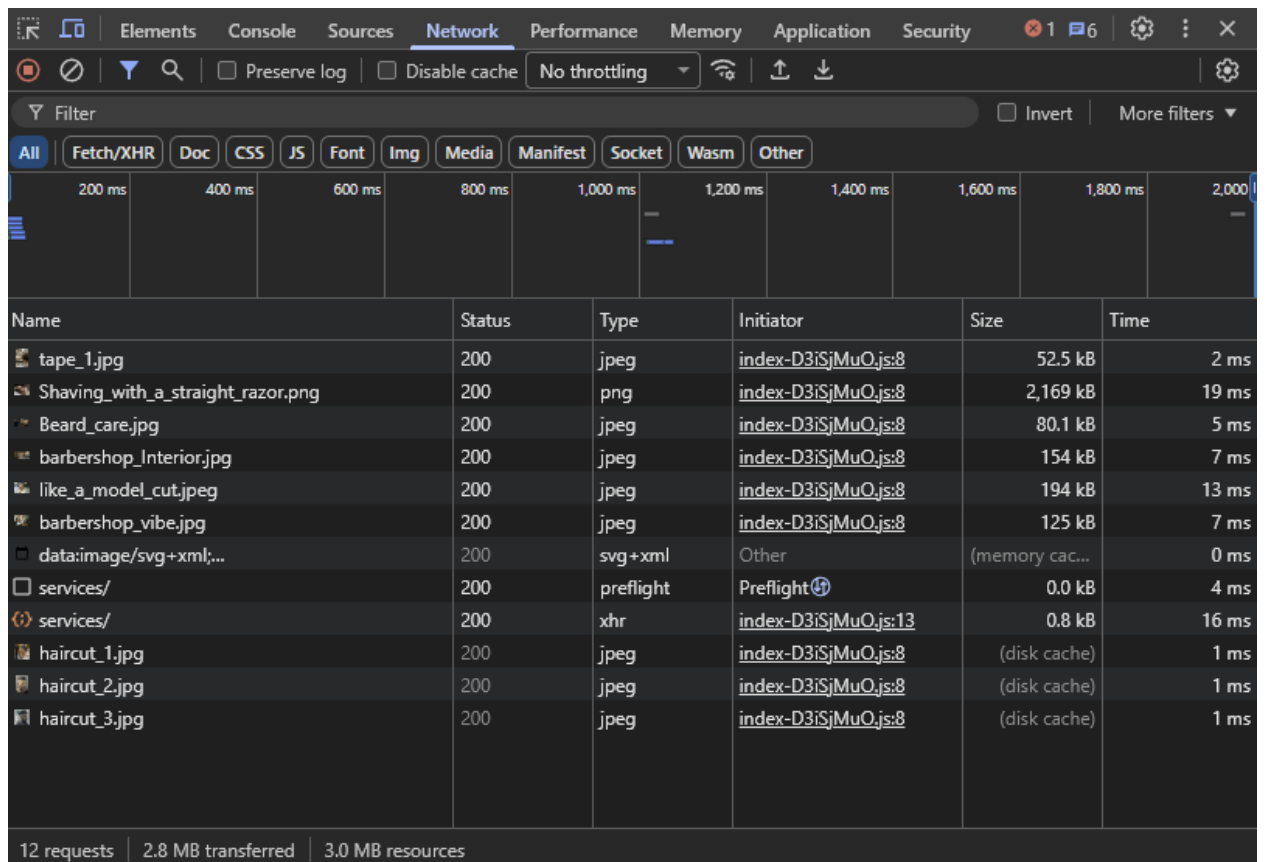
Нижний Новгород
2026

Ссылка на GitHub: <https://github.com/googlehater/barbershop>

Цель лабораторной работы

- Освоить клиентский рендеринг с помощью SPA
- Научиться использовать ORM
- Использовать архитектуру с разделением на клиент и сервер
- Научиться использовать авторизацию

1. Освоить клиентский рендеринг с помощью SPA

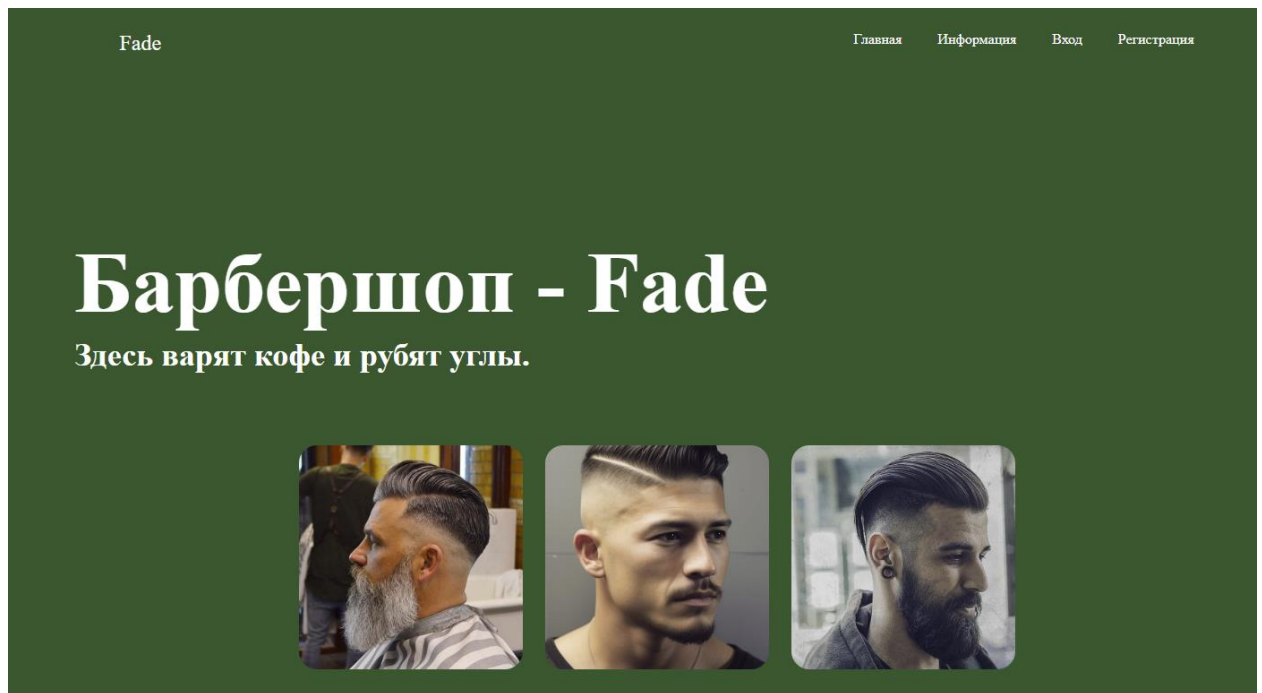


Name	Status	Type	Initiator	Size	Time
tape_1.jpg	200	jpeg	index-D3iSjMuO.js:8	52.5 kB	2 ms
Shaving_with_a_straight_razor.png	200	png	index-D3iSjMuO.js:8	2,169 kB	19 ms
Beard_care.jpg	200	jpeg	index-D3iSjMuO.js:8	80.1 kB	5 ms
barbershop_Interior.jpg	200	jpeg	index-D3iSjMuO.js:8	154 kB	7 ms
like_a_model_cut.jpeg	200	jpeg	index-D3iSjMuO.js:8	194 kB	13 ms
barbershop_vibe.jpg	200	jpeg	index-D3iSjMuO.js:8	125 kB	7 ms
data:image/svg+xml;...	200	svg+xml	Other	(memory cac...	0 ms
services/	200	preflight	Preflight	0.0 kB	4 ms
services/	200	xhr	index-D3iSjMuO.js:13	0.8 kB	16 ms
haircut_1.jpg	200	jpeg	index-D3iSjMuO.js:8	(disk cache)	1 ms
haircut_2.jpg	200	jpeg	index-D3iSjMuO.js:8	(disk cache)	1 ms
haircut_3.jpg	200	jpeg	index-D3iSjMuO.js:8	(disk cache)	1 ms

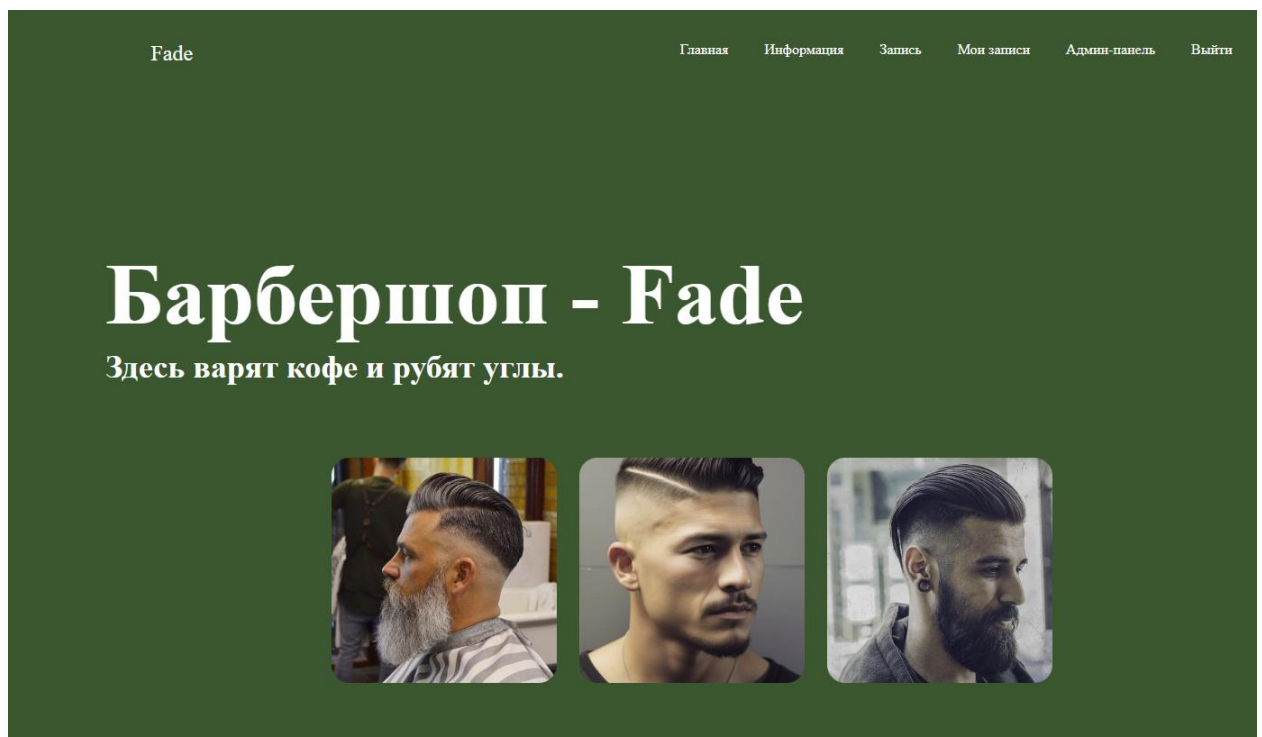
12 requests | 2.8 MB transferred | 3.0 MB resources

На скриншоте видно, что при переходе по ссылкам главная-инфоормация-вход нет никакой перезагрузки страницы. Меняется только контент. Во вкладке network нет никаких запросов на HTML.

Динамическое обновление интерфейса до логина:



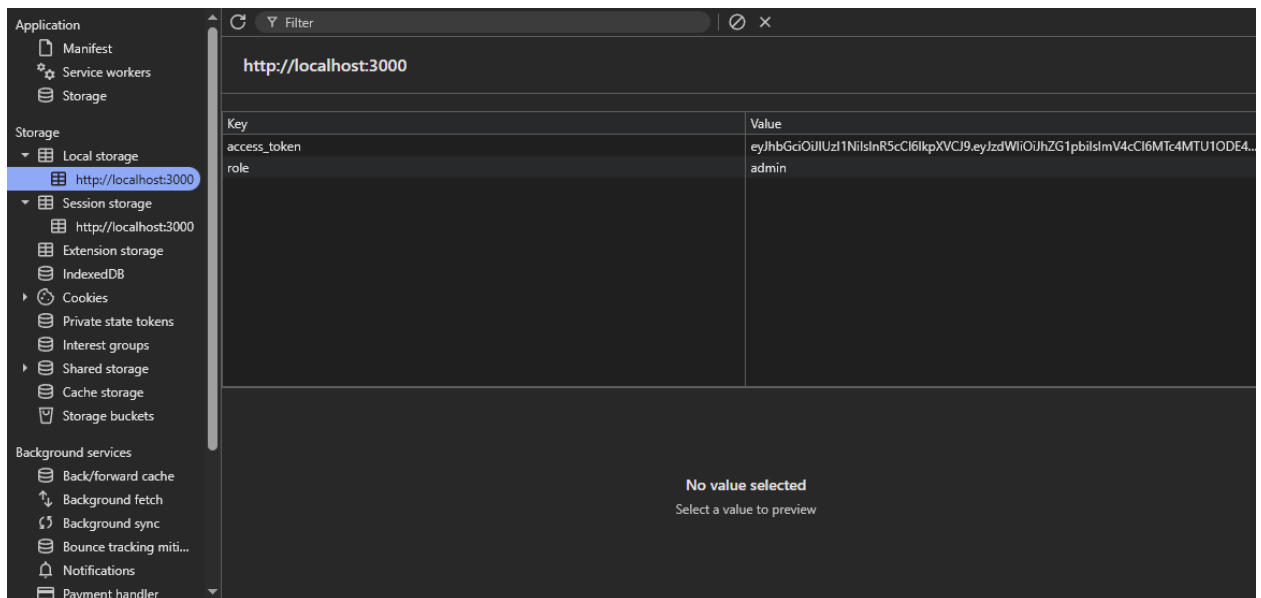
После:



Как видно, появилась админ – панель.

используется react-router-dom, который заменяет традиционную навигацию с полной перезагрузкой страницы на клиентскую. То есть можно с роута / перейти на /info

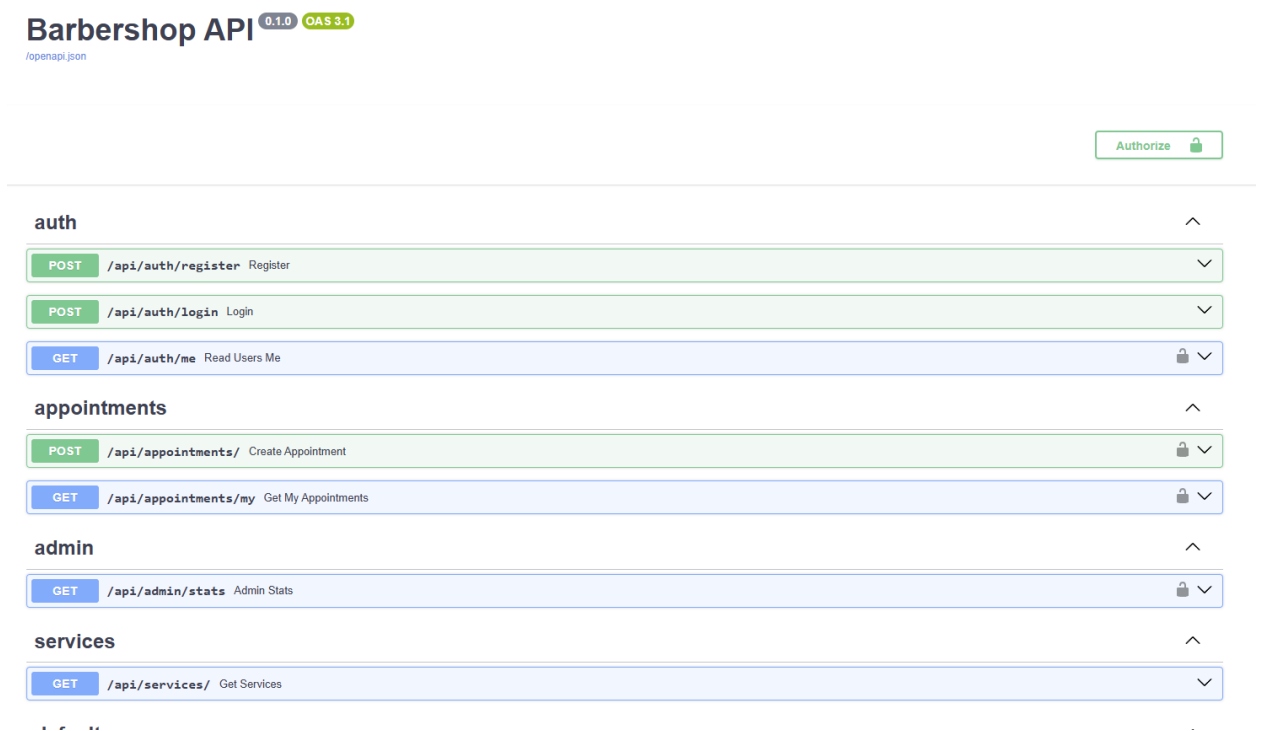
во вкладке **Local Storage**:



Видно, что есть access_token и role. Токен JWT сохраняется в браузере и автоматически подставляется в заголовки всех запросов

Также, при попытке зайти на роут /profile настроен редирект. Если пользователь не авторизован – редирект на /login. Это защита маршрутов. Требование «Аутентификация через JWT или сессии» выполнено и работает исправно

Также в качестве доказательства, что бекенд работает скриншот из свагера:



Авторизация пользователя aliice

Available authorizations



Scopes are used to grant an application different levels of access to data on behalf of the end user. Each API may declare one or more scopes.

API requires the following scopes. Select which ones you want to grant to Swagger UI.

OAuth2PasswordBearer (OAuth2, password)

Authorized

Token URL: /api/auth/login

Flow: password

username: alice

password: *****

Client credentials location: basic

client_secret: *****

Logout

Close

GET /api/auth/me Read Users Me

Parameters

No parameters

Execute Clear

Responses

Curl

```
curl -X 'GET' \
  'http://localhost:8000/api/auth/me' \
  -H 'accept: application/json' \
  -H 'Authorization: Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJzdWIiOiI1bG1jZSIsInV4cCI6IjE2MDA2MjB0.4Hpe9sFFw7Uo0L9HebQqKqJawwT7qAy3Vz9U2YEuf8'
```

Request URL

http://localhost:8000/api/auth/me

Server response

Code	Details
200	Response body <pre>{ "username": "alice", "email": "alice@gmail.com", "role": "user", "id": 3, "created_at": "2026-06-15T20:10:53.121363" }</pre> Response headers <pre>content-length: 109 content-type: application/json date: Mon, 15 Jun 2026 21:18:14 GMT server: uvicorn</pre>

Responses

Code	Description	Links
200	Successful Response	No links

Backend/app/services/auth_service.py

```

backend > app > services > auth_service.py > ...
1  import bcrypt
2  from jose import JWTError, jwt
3  from datetime import datetime, timedelta
4  from sqlalchemy.orm import Session
5  from app.models.user_model import User
6  from app.schemas.auth import TokenData
7  import os
8
9  SECRET_KEY = "your-secret-key-please-change-in-production"
10 ALGORITHM = "HS256"
11 ACCESS_TOKEN_EXPIRE_MINUTES = 30
12
13 def verify_password(plain_password: str, hashed_password: str) -> bool:
14     return bcrypt.checkpw(plain_password.encode('utf-8'), hashed_password.encode('utf-8'))
15
16 def get_password_hash(password: str) -> str:
17     salt = bcrypt.gensalt()
18     return bcrypt.hashpw(password.encode('utf-8'), salt).decode('utf-8')
19
20 def authenticate_user(db: Session, username: str, password: str):
21     user = db.query(User).filter(User.username == username).first()
22     if not user or not verify_password(password, user.password_hash):
23         return None
24     return user
25
26 def create_access_token(data: dict, expires_delta: timedelta | None = None):
27     to_encode = data.copy()
28     if expires_delta:
29         expire = datetime.utcnow() + expires_delta
30     else:
31         expire = datetime.utcnow() + timedelta(minutes=15)
32     to_encode.update({"exp": expire})
33     return jwt.encode(to_encode, SECRET_KEY, algorithm=ALGORITHM)
34
35 def get_current_user(db: Session, token: str):
36     try:
37         payload = jwt.decode(token, SECRET_KEY, algorithms=[ALGORITHM])
38         username: str = payload.get("sub")
39         if username is None:
40             return None
41         user = db.query(User).filter(User.username == username).first()
42         return user
43     except JWTError:
44         return None

```

- При хэшировании генерируется случайная **соль** (salt) – уникальная для каждого пароля
- Соль добавляется к паролю, затем применяется алгоритм шифрования
- Это делает невозможным использование радужных таблиц для подбора паролей
- Функция gensalt() может принимать параметр сложности (cost factor) – чем выше, тем медленнее подбор

Эндпоинт регистрации

Файл backend/app/api/auth_router.py:

```

12
13 router = APIRouter(prefix="/auth", tags=["auth"])
14 oauth2_scheme = OAuth2PasswordBearer(tokenUrl="/api/auth/login")
15
16 @router.post("/register", response_model=UserOut)
17 def register(user: UserCreate, db: Session = Depends(get_db)):
18     existing = db.query(User).filter(User.username == user.username).first()
19     if existing:
20         raise HTTPException(status_code=400, detail="Username already registered")
21     return create_user(db, user)
22
23 @router.post("/login")
24 def login(form_data: OAuth2PasswordRequestForm = Depends(), db: Session = Depends(get_db)):
25     user = authenticate_user(db, form_data.username, form_data.password)
26     if not user:
27         raise HTTPException(status_code=401, detail="Incorrect username or password")
28     access_token = create_access_token(
29         data={"sub": user.username},
30         expires_delta=timedelta(minutes=ACCESS_TOKEN_EXPIRE_MINUTES)
31     )
32     return {"access_token": access_token, "token_type": "bearer"}
33
34 @router.get("/me", response_model=UserOut)
35 def read_users_me(current_user: User = Depends(get_current_user)):
36     return current_user
37

```

Подключение к базе данных:

```

19 from sqlalchemy import create_engine
20 from sqlalchemy.orm import sessionmaker, Session
21 from app.models.base_model import Base
22
23 # ВНИМАНИЕ: для локальной разработки пока используем localhost
24 # Позже в Docker заменим на переменные окружения
25 DATABASE_URL = "postgresql://postgres:postgres@db:5432/barbershop"
26
27 engine = create_engine(DATABASE_URL, echo=True)
28 SessionLocal = sessionmaker(autocommit=False, autoflush=False, bind=engine)
29
30 def get_db() -> Session:
31     db = SessionLocal()
32     try:
33         yield db
34     finally:
35         db.close()
36

```

- create_engine() создаёт подключение к БД, параметр echo=True включает логирование всех SQL-запросов для отладки
- sessionmaker создаёт фабрику сессий – каждая сессия представляет собой "рабочую единицу" с БД

- `get_db()` – dependency для FastAPI, автоматически закрывает соединение после завершения запроса

Определение моделей (объявление таблиц):

backend/app/models/user.py

```
backend > app > models > user_model.py > User
1  from datetime import datetime
2
3  from sqlalchemy import String
4  from sqlalchemy import DateTime
5
6  from sqlalchemy.orm import Mapped
7  from sqlalchemy.orm import mapped_column, relationship
8
9  from app.models.base_model import Base
10
11
12  class User(Base):
13      __tablename__ = "users"
14
15      id: Mapped[int] = mapped_column(
16          primary_key=True
17      )
18
19      username: Mapped[str] = mapped_column(
20          String(50),
21          unique=True,
22          nullable=False
23      )
24
25      email: Mapped[str] = mapped_column(
26          String(255),
27          unique=True,
28          nullable=False
29      )
30
31      password_hash: Mapped[str] = mapped_column(
32          String(255),
33          nullable=False
34      )
35
36      role: Mapped[str] = mapped_column(
37          String(20),
38          default="user"
39      )
40
41      created_at: Mapped[datetime] = mapped_column(
42          DateTime,
43          default=datetime.utcnow
44      )
45
```

Таким образом описаны все сущности

- Каждый класс, наследующий Base, становится отдельной таблицей в БД
- `__tablename__` задаёт имя таблицы (если не указать, SQLAlchemy сгенерирует имя по имени класса)

- Column() определяет столбец с указанием типа (Integer, String, DateTime и др.)
- primary_key=True – первичный ключ
- ForeignKey() – внешний ключ (связь между таблицами)
- Enum – тип для полей с ограниченным набором значений
- relationship() – опционально, позволяет обращаться к связанным объектам как к атрибутам

Миграции Alembic:

Команды для работы с миграциями:

Создать новую миграцию на основе изменений в моделях

alembic revision --autogenerate -m "описание изменений"

Применить все неприменённые миграции

alembic upgrade head

Посмотреть текущую версию базы

alembic current

Откатить последнюю миграцию

alembic downgrade -1

пример сгенерированной миграции

#alembic/versions/209d3f5c1f47_initial_schema.py

```

def upgrade() -> None:
    """Upgrade schema."""
    # ### commands auto generated by Alembic - please adjust! ###
    op.create_table('masters',
        sa.Column('id', sa.Integer(), nullable=False),
        sa.Column('name', sa.String(length=100), nullable=False),
        sa.Column('phone', sa.String(length=20), nullable=True),
        sa.Column('average_score', sa.Numeric(precision=3, scale=2), nullable=True),
        sa.Column('works_since', sa.Date(), nullable=True),
        sa.Column('status', sa.Enum('fired', 'works', 'on_vacation', name='masterstatus'), nullable=True),
        sa.PrimaryKeyConstraint('id')
    )
    op.create_table('services',
        sa.Column('id', sa.Integer(), nullable=False),
        sa.Column('services_name', sa.String(length=100), nullable=False),
        sa.Column('duration_minutes', sa.Integer(), nullable=False),
        sa.Column('price', sa.Numeric(precision=10, scale=2), nullable=False),
        sa.Column('description', sa.Text(), nullable=True),
        sa.PrimaryKeyConstraint('id')
    )
    op.create_table('users',
        sa.Column('id', sa.Integer(), nullable=False),
        sa.Column('username', sa.String(length=50), nullable=False),
        sa.Column('email', sa.String(length=255), nullable=False),
        sa.Column('password_hash', sa.String(length=255), nullable=False),
        sa.Column('role', sa.String(length=20), nullable=False),
        sa.Column('created_at', sa.DateTime(), nullable=False),
        sa.PrimaryKeyConstraint('id'),
        sa.UniqueConstraint('email'),
        sa.UniqueConstraint('username')
    )

```

Также для теста того, что всё работает был создан файл seed.py с синтетическими данными. Чтобы его запустить нужно после поднятия backend контейнера выполнить команду `docker compose exec backend python seed.py`. Видно, что пункт «Для генерации схемы БД используются миграции» выполнен.

```

# 1. Создать админа и обычного пользователя, если их нет
admin = db.query(User).filter(User.username == "admin").first()
if not admin:
    admin = User(
        username="admin",
        email="admin@barbershop.com",
        password_hash=get_password_hash("admin123"),
        role="admin",
        created_at=datetime.utcnow()
    )
    db.add(admin)

user = db.query(User).filter(User.username == "john_doe").first()
if not user:
    user = User(
        username="john_doe",
        email="john@example.com",
        password_hash=get_password_hash("password123"),
        role="user",
        created_at=datetime.utcnow()
    )
    db.add(user)

db.commit()
print("✓ Пользователи добавлены")

# 2. Услуги
services_data = [
    {"services_name": "Стрижка", "duration_minutes": 30, "price": 1000, "description": "Стрижка"},
    {"services_name": "Бритьё", "duration_minutes": 20, "price": 800, "description": "Бритьё"},
    {"services_name": "Укладка", "duration_minutes": 25, "price": 700, "description": "Укладка"},
    {"services_name": "Стрижка + борода", "duration_minutes": 50, "price": 1600, "description": "Стрижка + борода"}
]
for s in services_data:
    if not db.query(Service).filter(Service.services_name == s["services_name"]).first():
        db.add(Service(**s))
db.commit()

```

Логи:

```

FROM users
WHERE users.username = %(username_1)s
LIMIT %(param_1)s
2026-06-15 19:42:19,270 INFO sqlalchemy.engine.Engine [generated in 0.00016s] {'username_1': 'admin', 'param_1': 1}
2026-06-15 19:42:19,273 INFO sqlalchemy.engine.Engine SELECT users.id AS users_id, users.username AS users_username, users.email AS users_email, users.password_hash AS users_password_hash, users.role AS users_role, users.created_at AS users_created_at
FROM users
WHERE users.username = %(username_1)s
LIMIT %(param_1)s
2026-06-15 19:42:19,273 INFO sqlalchemy.engine.Engine [cached since 0.004002s ago] {'username_1': 'john_doe', 'param_1': 1}
2026-06-15 19:42:19,274 INFO sqlalchemy.engine.Engine COMMIT
✓ Пользователи добавлены
2026-06-15 19:42:19,275 INFO sqlalchemy.engine.Engine BEGIN (implicit)
2026-06-15 19:42:19,276 INFO sqlalchemy.engine.Engine SELECT services.id AS services_id, services.services_name AS services_services_name, services.duration_minutes AS services_duration_minutes, services.price AS services_price, services.description AS services_description
FROM services
WHERE services.services_name = %(services_name_1)s
LIMIT %(param_1)s
2026-06-15 19:42:19,276 INFO sqlalchemy.engine.Engine [generated in 0.00016s] {'services_name_1': 'Стрижка', 'param_1': 1}
2026-06-15 19:42:19,277 INFO sqlalchemy.engine.Engine SELECT services.id AS services_id, services.services_name AS services_services_name, services.duration_minutes AS services_duration_minutes, services.price AS services_price, services.description AS services_description
FROM services
WHERE services.services_name = %(services_name_1)s
LIMIT %(param_1)s
2026-06-15 19:42:19,277 INFO sqlalchemy.engine.Engine [cached since 0.001697s ago] {'services_name_1': 'Бритьё', 'param_1': 1}
2026-06-15 19:42:19,278 INFO sqlalchemy.engine.Engine SELECT services.id AS services

```

Тут видно, что sqlalchemy работает, а данные заполняются. Также видно, что выполняется пункт «Все запросы в БД происходят строго через ORM»

Запись в бд:

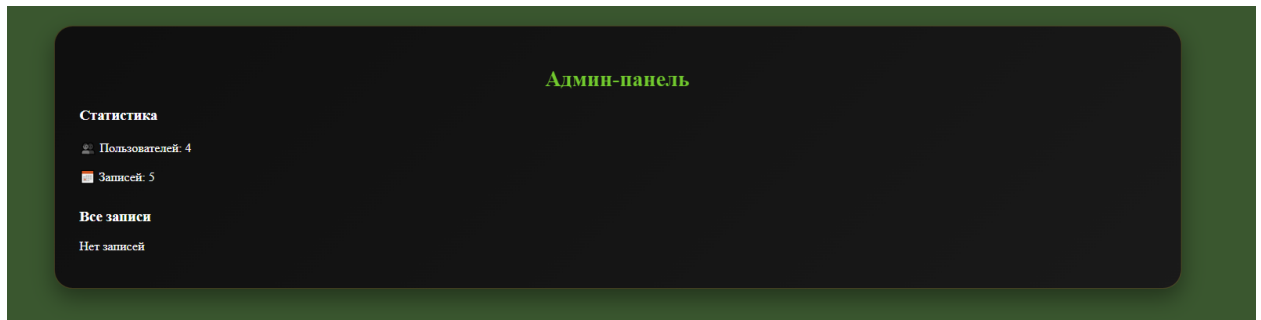
```

(venv) PS D:\NGTU\3_c\LP> docker compose exec db psql -U postgres -d barbershop -c "
SELECT username, role FROM users;"
username | role
-----+-----
admin    | admin
john_doe | user
alice    | user
ugi      | user
(4 rows)

```

Пользователь ugi был создан через рут /register

«Контент страницы зависит от прав пользователя. Можно предоставлять данные которые ему принадлежат или давать доступ к разным страницам в зависимости от прав. Самое банальное - админская панель»



Теперь про использование SPA фреймворков:

В качестве клиентского фреймворка выбран **React** (версия 20, потому что vite требовал 20-alpine+, но сначала брал 18) в связке со сборщиком **Vite**. Выбор обоснован следующими преимуществами:

- **Компонентный подход** – позволяет создавать переиспользуемые UI-блоки (Navbar, Footer, формы)
- **Виртуальный DOM** – обеспечивает высокую производительность при обновлении интерфейса
- **JSX синтаксис** – упрощает написание шаблонов, сочетая HTML и JavaScript
- **Vite** – даёт быструю холодную сборку и мгновенную горячую перезагрузку (HMR)
- **React Router DOM** – обеспечивает клиентскую маршрутизацию без перезагрузки страницы

Фронтенд-приложение работает как **Single Page Application (SPA)**. Все переходы между страницами происходят без полной перезагрузки браузера, только динамически заменяется содержимое контейнера #root. Это достигается за счёт использования библиотеки react-router-dom.

Файл main.jsx

```
import { StrictMode } from 'react'
import { createRoot } from 'react-dom/client'
import './styles/global.css'
import App from './App.jsx'

createRoot(document.getElementById('root')).render(
  <StrictMode>
    <App />
  </StrictMode>,
)
```

Маршрутизация реализована через компоненты BrowserRouter, Routes и Route. При изменении URL браузер не отправляет запрос на сервер – React Router перехватывает навигацию и рендерит соответствующий компонент.

App.jsx:

```
frontend > src > App.jsx > App
1  import { BrowserRouter, Routes, Route } from 'react-router-dom';
2  import Home from './pages/Home';
3  import About from './pages/About';
4  import Appointment from './pages/Appointment';
5  import Login from './pages/Login';
6  import Register from './pages/Register';
7  import Profile from './pages/Profile';
8  import Admin from './pages/Admin';
9  import PrivateRoute from './components/PrivateRoute';
10 import AdminRoute from './components/AdminRoute';
11 import Navbar from './components/Navbar';
12 import Footer from './components/Footer';
13
14 function App() {
15   return (
16     <BrowserRouter>
17       <Navbar />
18       <Routes>
19         <Route path="/" element={<Home />} />
20         <Route path="/about" element={<About />} />
21         <Route path="/login" element={<Login />} />
22         <Route path="/register" element={<Register />} />
23         <Route element={<PrivateRoute />>
24           <Route path="/appointment" element={<Appointment />} />
25           <Route path="/profile" element={<Profile />} />
26         </Route>
27         <Route element={<AdminRoute />>
28           <Route path="/admin" element={<Admin />} />
29         </Route>
30       </Routes>
31       <Footer />
32     </BrowserRouter>
33   );
34 }
35 export default App;
36
```

«Должно быть явное разделение на фронтенд и бэкенд, у каждого свой контейнер»:

```

🐳 docker-compose.yml
1  services:
2    db:
3      image: postgres:17-alpine
4      environment:
5        POSTGRES_USER: ${DB_USER}
6        POSTGRES_PASSWORD: ${DB_PASSWORD}
7        POSTGRES_DB: ${DB_NAME}
8      ports:
9        - "${DB_PORT}:5432"
10     volumes:
11       - pgdata:/var/lib/postgresql/data
12       # - ./src/database:/docker-entrypoint-initdb.d
13     healthcheck:
14       test: ["CMD-SHELL", "pg_isready -U ${DB_USER} -d ${DB_NAME}"]
15       interval: 5s
16       timeout: 5s
17       retries: 5
18
19     backend:
20       build:
21         context: ./backend
22         dockerfile: Dockerfile
23       environment:
24         DB_HOST: db
25         DB_PORT: 5432
26         DB_USER: ${DB_USER}
27         DB_PASSWORD: ${DB_PASSWORD}
28         DB_NAME: ${DB_NAME}
29         DATABASE_URL: postgresql://${DB_USER}:${DB_PASSWORD}@db:5432/${DB_NAME}
30         # FLASK_ENV: development
31         # FLASKDEBUG: 1
32       ports:
33         - "8000:8000"
34       volumes:
35         - ./backend:/app # all folder prjct
36         - /app/__pycache__ # cache ignore
37       depends_on:
38         db:
39           condition: service_healthy
40       restart: on-failure
41       # working_dir: /app
42
43     frontend:
44       build: ./frontend
45       ports:
46         - "3000:80"
47       depends_on:
48         - backend
49

```

Также структура подразумевает явное разделение:

.git	05.06.2026 16:10	Папка с файлами	
__pycache__	12.05.2026 0:25	Папка с файлами	
backend	13.06.2026 23:00	Папка с файлами	
docs	16.06.2026 0:22	Папка с файлами	
frontend	15.06.2026 17:42	Папка с файлами	
src	07.05.2026 14:51	Папка с файлами	
venv	13.06.2026 11:40	Папка с файлами	
.env	14.06.2026 14:26	Файл ".ENV"	1 КБ
.env_example	13.05.2026 0:40	Файл ".ENV_EXAM...	1 КБ
.gitignore	07.05.2026 15:25	Текстовый докум...	1 КБ
docker-compose.yaml	15.06.2026 17:24	Исходный файл ...	2 КБ
README.md	15.06.2026 23:14	Исходный файл ...	4 КБ
README_RUS.md	13.05.2026 1:05	Исходный файл ...	3 КБ
to_start.txt	13.06.2026 23:06	Текстовый докум...	1 КБ
tree.txt	13.06.2026 12:22	Текстовый докум...	26 КБ

Для HTTP-запросов используется библиотека `axios`. Создан централизованный клиент с перехватчиком запросов, который автоматически добавляет JWT-токен в заголовок `Authorization`

У каждой части приложения есть свой `Dockerfile`:

Backend:

```
FROM python:3.13-slim

WORKDIR /app

# Устанавливаем системные зависимости (для psycopg2)
RUN apt-get update && apt-get install -y libpq-dev gcc && rm -rf /var/lib/apt/lists/*

# Копируем зависимости и устанавливаем их
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

# Копируем весь код бэкенда
COPY . .

# Переменные окружения (могут быть переопределены в compose)
ENV PYTHONPATH=/app

# Команда для запуска FastAPI (через uvicorn)
CMD ["uvicorn", "app.main:app", "--host", "0.0.0.0", "--port", "8000"]
```

Frontend:

```

nd > Dockerfile
FROM node:20-alpine AS build
WORKDIR /app
COPY package*.json ./
RUN npm install
COPY . .
RUN npm run build

FROM nginx:alpine
COPY --from=build /app/dist /usr/share/nginx/html
COPY nginx.conf /etc/nginx/conf.d/default.conf
EXPOSE 80
CMD ["nginx", "-g", "daemon off;"]

```

Взаимодействие бэкенда и фронтенда через api, api соответствует правилам REST

REST (Representational State Transfer) – архитектурный стиль для построения веб-API, основанный на некоторых принципах

Так в проекте есть эндпоинты API:

/api/auth/register - POST Регистрация нового пользователя, не требует авторизации

/api/auth/login - POST Вход в систему (получение JWT), не требует авторизации

/api/auth/me – GET Получение данных текущего пользователя, требует JWT

Услуги:

/api/services/- GET Получение списка всех услуг, не требует авторизации

/api/services/{id} - GET Получение услуги по ID, не требует авторизации

мастера:

/api/appointments/my - GET Получение записей текущего пользователя, требует авторизации JWT

/api/appointments/- POST Создание новой записи, требует авторизации JWT

/api/appointments/- GET Получение всех записей (только админ), требует авторизации JWT

Админ-панель (Admin):

/api/admin/stats- GET Получение всех записей (только админ), требует авторизации JWT

Примеры ответов на запросы:

Curl

```
curl -X 'GET' \
  'http://localhost:8000/api/services/' \
  -H 'accept: application/json'
```

Request URL

http://localhost:8000/api/services/

Server response

Code	Details
200	Response body <pre>[{ "duration_minutes": 30, "services_name": "Стрижка", "description": "Классическая стрижка", "id": 1, "price": 1000 }, { "duration_minutes": 20, "services_name": "Бритьё", "description": "Бритьё опасной бритвой", "id": 2, "price": 800 }, { "duration_minutes": 25, "services_name": "Укладка", "description": "Укладка воском/гелем", "id": 3, "price": 700 }, { "duration_minutes": 50, "services_name": "Стрижка + борода", "description": "Комплекс", "id": 4, "price": 1600 }]</pre> Download

Curl

```
curl -X 'GET' \
  'http://localhost:8000/api/admin/stats' \
  -H 'accept: application/json' \
  -H 'Authorization: Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJzdWIiOiI1MzG1pb1IsImV4cCI6MTU2MDQ0MD0.76Ma2c3Vg1HCxqhmKtkmYqBTsn-I-ZHZER7dtSeBc10'
```

Request URL

http://localhost:8000/api/admin/stats

Server response

Code	Details
200	Response body <pre>{ "total_users": 4, "total_appointments": 5 }</pre> Download Response headers <pre>content-length: 40 content-type: application/json date: Mon, 15 Jun 2026 21:24:10 GMT server: uvicorn</pre>

Responses

В бекенде:

```

router = APIRouter(prefix="/auth", tags=["auth"])
oauth2_scheme = OAuth2PasswordBearer(tokenUrl="/api/auth/login")

@router.post("/register", response_model=UserOut)
def register(user: UserCreate, db: Session = Depends(get_db)):
    existing = db.query(User).filter(User.username == user.username).first()
    if existing:
        raise HTTPException(status_code=400, detail="Username already registered")
    return create_user(db, user)

@router.post("/login")
def login(form_data: OAuth2PasswordRequestForm = Depends(), db: Session = Depends(get_db)):
    user = authenticate_user(db, form_data.username, form_data.password)
    if not user:
        raise HTTPException(status_code=401, detail="Incorrect username or password")
    access_token = create_access_token(
        data={"sub": user.username},
        expires_delta=timedelta(minutes=ACCESS_TOKEN_EXPIRE_MINUTES)
    )
    return {"access_token": access_token, "token_type": "bearer"}

@router.get("/me", response_model=UserOut)
def read_users_me(current_user: User = Depends(get_current_user)):
    return current_user

```

Реализация на фронтенде:

```

frontend > src > services > JS api.js > ...
1  import axios from 'axios';
2  const api = axios.create({ baseURL: 'http://localhost:8000/api' });
3  api.interceptors.request.use(config => {
4      const token = localStorage.getItem('access_token');
5      if (token) config.headers.Authorization = `Bearer ${token}`;
6      return config;
7  });
8  export default api;
9

```

Правка:

Замена document.getElementById* на хук useRef.

```

frontend > src > pages > Home.jsx
1  import { useEffect, useRef, useState } from 'react';
2
3  export default function Home() {
4      const [isDark, setIsDark] = useState(() => {
5          return localStorage.getItem('theme') === 'dark';
6      });
7      const toggleBtnRef = useRef(null);
8
9      useEffect(() => {
10         const body = document.body;
11         if (isDark) {
12             body.classList.add('dark-theme');
13             if (toggleBtnRef.current) toggleBtnRef.current.textContent = '☀ Светлая тема';
14         } else {
15             body.classList.remove('dark-theme');
16             if (toggleBtnRef.current) toggleBtnRef.current.textContent = '🌙 Тёмная тема';
17         }
18         localStorage.setItem('theme', isDark ? 'dark' : 'light');
19     }, [isDark]);
20
21     const toggleTheme = () => {
22         setIsDark(!isDark);
23     };
24
25     return (
26         <button className="theme-toggle" ref={toggleBtnRef} onClick={toggleTheme}>
27             🌙 Тёмная тема
28         </button>
29     );
30 }
31

```

Больше не было обнаружено использования `document.getElement*` или `document.query*`.

Правка:

Реализовать сервисный слой через объекты

При первой версии проекта, была совершена ошибка (автор забыл) и всё было реализовано через функции, что ведет к слабой связности и отсутствие DI.

Исправленные версии сервисного слоя:

```

9
10 class UserService:
11     def __init__(self, db: Session):
12         self.db = db
13         self.auth_service = AuthService(db)
14
15     def register(self, user_data: UserCreate):
16         existing = self.db.query(User).filter(User.username == user_data.username).first()
17         if existing:
18             raise HTTPException(status_code=400, detail="Username already registered")
19
20         hashed_password = self.auth_service.get_password_hash(user_data.password)
21         db_user = User(
22             username=user_data.username,
23             email=user_data.email,
24             password_hash=hashed_password,
25             role=user_data.role,
26             created_at=datetime.utcnow()
27         )
28         self.db.add(db_user)
29         self.db.commit()
30         self.db.refresh(db_user)
31         return db_user
32

```

Правка:

Нужно вынести из контроллеров лишнюю логику, все вызовы с ORM тоже вынести

```

backend > app > api > auth_router.py > ...
14
15     router = APIRouter(prefix="/auth", tags=["auth"])
16     oauth2_scheme = OAuth2PasswordBearer(tokenUrl="/api/auth/login")
17
18     @router.post("/register", response_model=UserOut)
19     def register(
20         user: UserCreate,
21         user_service: UserService = Depends(UserService),
22     ):
23         return user_service.register(user)
24
25     @router.post("/login")
26     def login(
27         form_data: OAuth2PasswordRequestForm = Depends(),
28         auth_service: AuthService = Depends(AuthService),
29     ):
30         user = auth_service.authenticate_user(form_data.username, form_data.password)
31         if not user:
32             raise HTTPException(status_code=401, detail="Incorrect username or password")
33         access_token = auth_service.create_access_token(data={"sub": user.username})
34         return {"access_token": access_token, "token_type": "bearer"}
35
36     @router.get("/me", response_model=UserOut)
37     def read_users_me(
38         token: str = Depends(oauth2_scheme),
39         auth_service: AuthService = Depends(AuthService),
40     ):
41         user = auth_service.get_current_user(token)
42         if not user:
43             raise HTTPException(status_code=401, detail="Invalid token")
44         return user
45

```

Нет никакой лишней логики, только исполнение команд. Лишняя логика происходит «под капотом»

```

backend > app > api > appointments_router.py > ...
1  from fastapi import APIRouter, Depends, HTTPException
2  from app.services.appointment_service import AppointmentService
3  from app.schemas.appointment import AppointmentCreate, AppointmentOut
4  from app.models.user_model import User
5  from app.api.deps import get_current_user
6
7  router = APIRouter(prefix="/appointments", tags=["appointments"])
8
9  @router.post("/", response_model=AppointmentOut)
10 def create_appointment(
11     apt: AppointmentCreate,
12     current_user: User = Depends(get_current_user),
13     appointment_service: AppointmentService = Depends(AppointmentService),
14 ):
15     return appointment_service.create_appointment(current_user.id, apt)
16
17 @router.get("/my", response_model=list[AppointmentOut])
18 def get_my_appointments(
19     current_user: User = Depends(get_current_user),
20     appointment_service: AppointmentService = Depends(AppointmentService),
21 ):
22     return appointment_service.get_user_appointments(current_user.id)
23

```

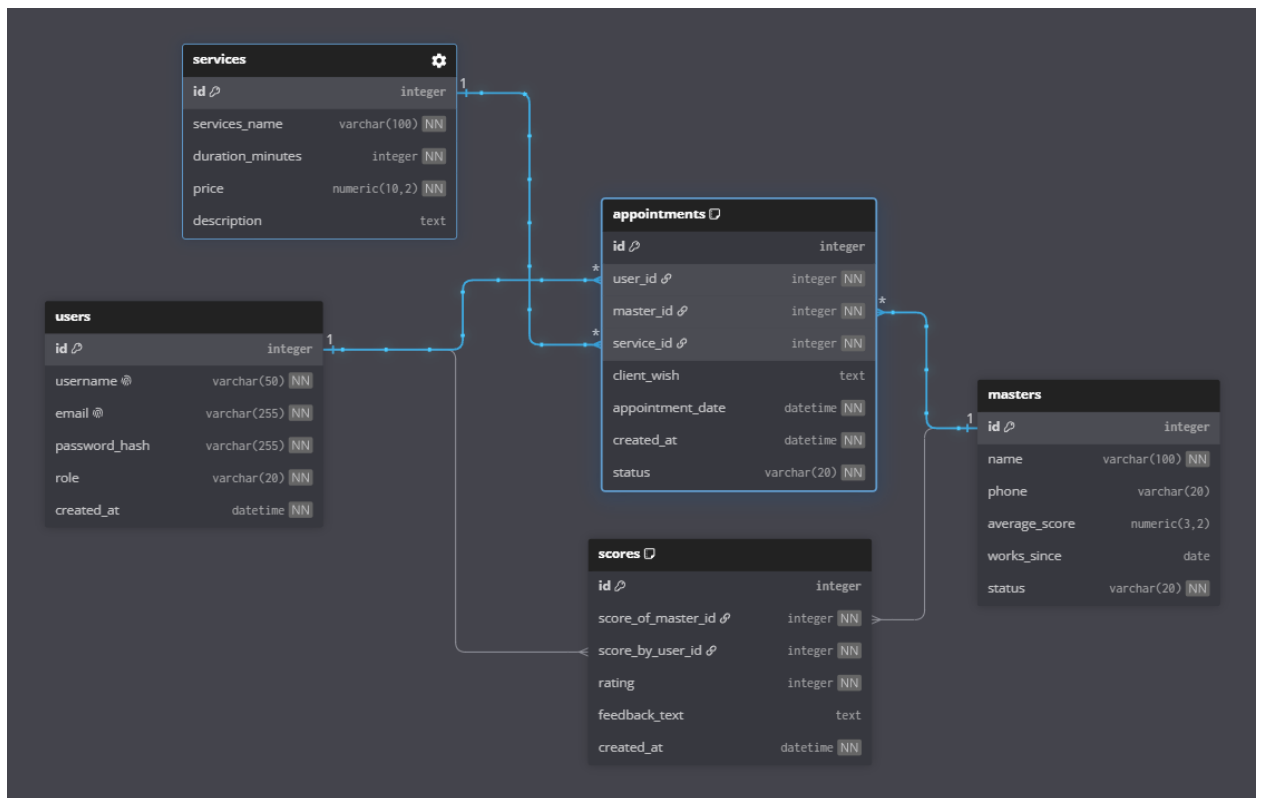
```

backend > app > api > admin_router.py > ...
1  from fastapi import APIRouter, Depends, HTTPException
2  from app.services.admin_service import AdminService
3  from app.models.user_model import User
4  from app.api.deps import get_current_user
5  from app.core.database import get_db
6
7  router = APIRouter(prefix="/admin", tags=["admin"])
8
9  def get_admin_service(db: Session = Depends(get_db)):
10     return AdminService(db)
11
12 @router.get("/stats")
13 def admin_stats(
14     current_user: User = Depends(get_current_user),
15     admin_service: AdminService = Depends(get_admin_service),
16 ):
17     if current_user.role != "admin":
18         raise HTTPException(status_code=403, detail="Not enough permissions")
19     return admin_service.get_stats()
20

```

Всё исправлено.

ERD-диаграмма:



Также была исправлена ошибка с передачей DATABASE_URL не через .env:

```

os.load_dotenv()

engine = create_engine(os.getenv("DATABASE_URL"), echo=True)
SessionLocal = sessionmaker(autocommit=False, autoflush=False, bind=engine)

def get_db() -> Session:
    db = SessionLocal()
    try:
        yield db
    finally:
        db.close()
  
```

Вывод:

В ходе выполнения лабораторной работы был разработан полнофункциональный веб-проект с использованием современных технологий:

Бэкенд построен на FastAPI с использованием SQLAlchemy ORM для работы с PostgreSQL. Миграции управляются через Alembic, так что можно использовать контроль версий БД и в курьезном случае откатиться к старой версии.

Фронтенд реализован как SPA на React с клиентским роутингом, что обеспечивает быстрый отклик интерфейса без перезагрузки страниц.

Аутентификация построена на JWT-токенах с хэшированием паролей через bcrypt. Вынесена отдельно

Разграничение прав реализовано через роли пользователей (user и admin), что позволяет контролировать доступ к контенту. Админ может смотреть статистику, добавлять или удалять контент, обычный пользователь не может этого

Деплой автоматизирован через Docker Compose и два файла Dockerfile в бекенде и фронтенде.